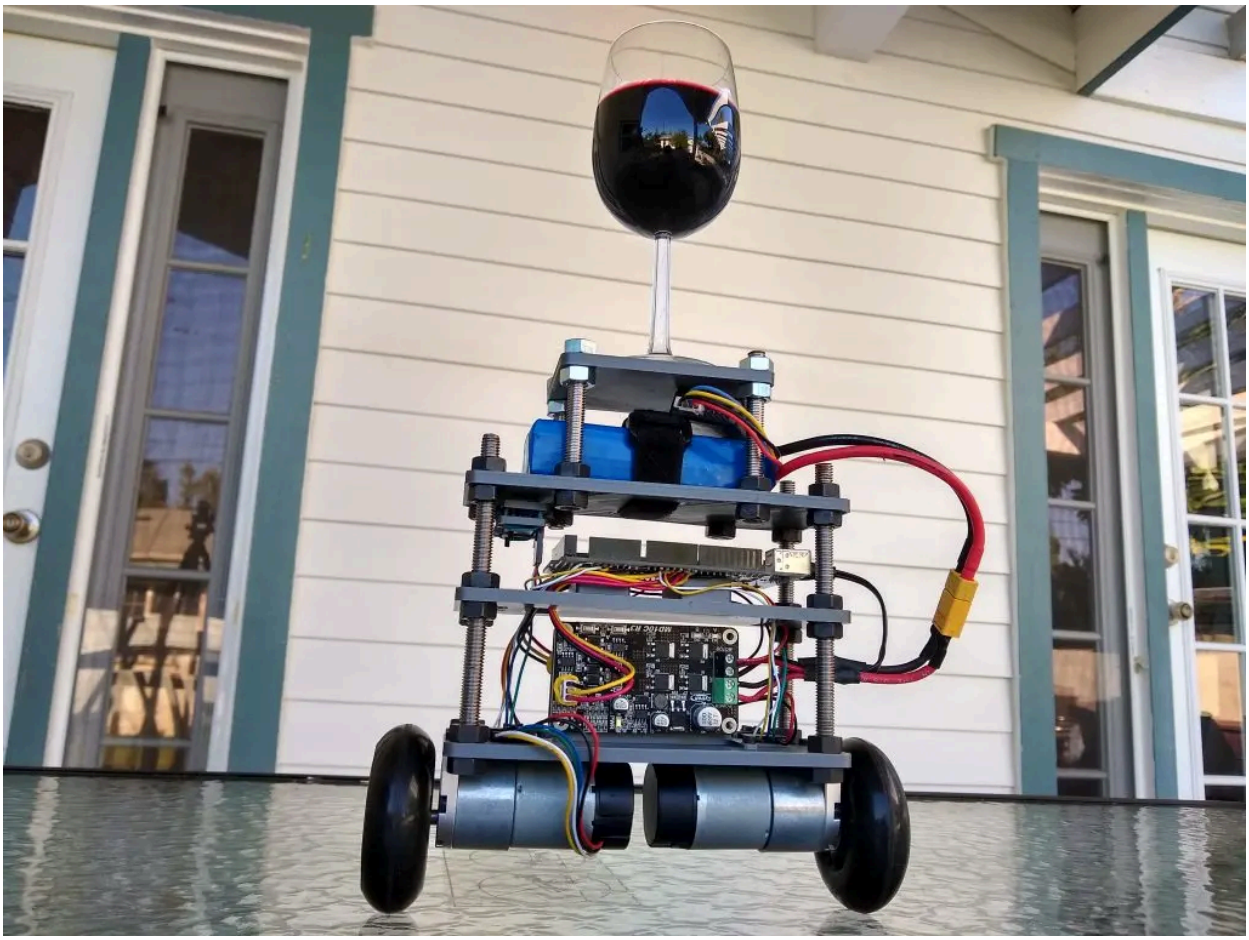
Shay Sackett's Project Portfolio

Self-Balancing Inverted Pendulum Robot

7 Comments / Uncategorized / By Shay Sackett



Contents [hide]

Overview:

This is a self-balancing, inverted pendulum robot that I built and programmed! The final result you see above is the result of the dedication of about a year's worth of work in my free time outside of school and a failed initial prototype. In this blog post I will go into way too much detail on the design, programming, and challenges that I faced and overcame in this project, but if you don't want to read the whole thing no worries! After the video below I've written a quick summary to explain the basic technical specifications and capabilities of this robot.

I programmed this robot using the Arduino language and an Arduino Mega 2560. I used two 30:1 brushed DC gearmotors from Pololu each with 64 counts per revolution (CPR) quadrature encoders. To sense orientation the robot uses an MPU-6050 IMU, and the whole robot is powered by a 3-cell LiPo battery on the top "shelf". The entire robot code loop runs 250 times per second, precisely every 4 milliseconds. The robot uses a complimentary filter to combine the accelerometer and gyroscope data from the MPU-6050 IMU into an accurate measure of the angle of the robot in relation to the ground. The angle of the robot is then fed into one of two nested PID algorithms, the first algorithm of which controls the speed of the motors and works to drive the angle of the robot to an angle commanded by a second PID algorithm. The second PID algorithm is fed the velocity of the robot from the encoders (a piece of code I derived myself and while it's nothing groundbreaking I am quite proud of it) and works to bring the velocity of the robot to zero by commanding . These two PID algorithms working in concert is what allows this robot to carry a glass of wine dynamically, or other objects balanced haphazardly on top of the robot without needing to adjust any robot parameters. Both PID algorithms work together to find a "stable" resting configuration for the robot. Pretty cool huh? If you're still want to know more, keep reading because I have written A LOT on this project!

Project Inspirations:

Why did I decide to build a self-balancing robot at all?

The impetus for this project grew out of my days of flying quadcopters and tricopters. I was curious as to how all the various sensor data on my multitrotors was fed into the flight computer and how the flight computer could keep the aircraft stable and flying despite headwinds or even someone pulling on the multirotor itself. How does a quadcopter combine all the data from the accelerometer, gyroscope, GPS, and other sensors into a correct position estimate? These are all very different sensors with different strengths, weaknesses, and best use cases, so how does the multirotor "know", how and "when", to use each sensor? These were all questions I was pondering and wanted answers to.

I had a couple other inspirations for this project. The first was the mind-blowing "Cubli" video. Another inspiration came from a video for a flat plate that could balance a ball bearing on its surface, and move the ball around without losing control and dropping the ball off the plate. The final inspiration was all the videos and livestreams of SpaceX landing their Falcon 9 rockets vertically at sea after successfully delivering a satellite to space. I mean, what kind of programming-math wizardry does it take to bring back a ten-story tall rocket back from space and land it vertically on a floating, tipping and heaving barge at sea!?!?!? I wanted to know!!!

My Interest in Control Theory:

This all led me to the field of classical control theory which I rapidly became interested in. I was fascinated by the idea of controlling a machine or robot and with the right algorithms allowing it to stay in a state of unnatural, unstable equilibrium. I eventually decided to design, build, and program a 2 wheeled balancing robot not unlike a Segway which I hoped would shed light on some of the mysteries of control theory and give me experience with programming. I didn't have much programming experience before this project, so it really was a daunting, "in over my head" feeling when I just decided to dive in and hope I could learn the necessary skills to make a robot balance. I'm glad I did just jump in though, because while I felt the chances of me failing were rather high, in the end I ended up learning so much more than if I had gone for a less ambitious project!

My First (Failed) Robot Prototype:

Skipping ahead, after some design work with Autodesk Inventor and struggles with my community college's old Dimension 3D printer here was my first result:

The
wiring
was a
bit
snug

Building a Brushless Motor Balancing Robot:

My first attempt at a balancing robot was very different than what I ultimately ended up with. Initially, I wanted to use brushless motors to power the robot. Coming from my experience with tricopters and quadcopters I was super impressed by the potential of brushless motors, and with the small compact power of brushless gimbal motors I wanted to build a small, sleek balancing robot. I repurposed a RCTimer V1.0 gimbal board I had from a brushless 2-axis GoPro gimbal I designed for my tricopter to be the main control board for my robot since the RCTimer board already had all the necessary components for a balancing robot. The RCTimer board had 2 brushless motor drivers, an Arduino compatible Atmel 328P that I could program with the standard Arduino IDE, and a MPU-6050 inertial measurement unit (IMU) for detecting the orientation of the robot. I used a small three cell LiPo laying around to power the robot and I used Lego Mindstorm tires for the wheels.

RCTimer v1.0 board at left, and MPU-6050 inertial measurement unit at right.

My Difficulties in Programming Brushless Motors at Low Speeds:

Sine PWM, bit-shift operations, low-level control, oh my!

However I quickly discovered programming brushless motors to turn at the very slow speeds required by a balancing robot without encoders turned out to be much more difficult than I thought. It turns out to drive a brushless motor at slow speeds one needs to use Sine PWM (SPWM) or Space Vector PWM (SVPWM). A guy named “Matt” online, the same one who built the balancing ball platform above, also built a brushless motor balancing robot, and I was using his code as reference to see what it would take to get a robot to balance. Initially I took a lot of inspiration from his robot design wise too, as you can see I pretty much designed my robot with the exact same form factor as his. Good artists copy great artists steal amirite??? Anyways what I was trying to get at was that Matt’s code was WAY beyond my beginner experience. There were all sorts of operators and things going on I didn’t recognize at first, examples being bit-shifting and Arduino hardware timer usage, as well as interrupt service routines, all of which were well beyond me. I spent my whole summer that year just trying to reverse engineer Matt’s code and learn what was going on. I never understand it 100%, but I did get pretty close, and I learned A LOT picking through his code, and even filled an entire notebook worth of notes. I learned how to use bitshift operators for one, and I at least became familiar with SPWM and SVPWM in principle, even if I never understood how to implement them in code at the time. As an aside I did come back to controlling brushless motors at slow speeds a few years later. To wrap it up, at

a certain point I realized I was spending all this time trying to learn how to control brushless motors, and I hadn't even gotten to the interesting problems of trying to balance a robot! At the time, the best I managed to do with controlling those brushless motors was to make them vibrate, whine, and get really, burning hot to the touch, which is not the effect I was looking for.

A disassembled example of one of the two brushless motors I tried to turn into a balancing robot

My Second, Successful, Attempt at A Balancing Robot

I went back to the drawing board an realized brushless motors were not the way to go for me.

And so in the interest of actually solving the problem of balancing a robot, I went back to the drawing board. I decided to go back to the larger, less efficient, but tried and true brushed DC motors. Once I made the decision to go back to brushed DC motors the design went pretty smoothly. I chose to go with a "stacked" robot design, where different components were placed on top of one another on "shelves" or "levels". I also chose a much more conventional electronics layout, with 2 standard brushed DC Motor controllers and an Arduino Mega 2560 at the heart of the robot.

My initial CAD Design of the Robot

Free-Floating the MPU-6050 Inertial Measurement Unit:

The MPU-6050 IMU had done me no wrongs on the previous robot, so I decided to use it here as well, with the additional caveat that I went to extra lengths to insulate the inertial measurement unit from vibrations caused by the motors, or someone (like me), stomping around the robot while it was in motion. So I designed a probably over-engineered but (I think) slick little vibration damping mechanism using moon gel. Moon gel is generally used by drummers to tune the sound of drums, as it absorbs vibrations incredibly well. I initially used moon gel on my tricopter and quadcopter to isolate the flight CPU from motor vibrations and it did a brilliant job there so I reused it to dampen the IMU vibrations here as well. Thanks to moon gel and this vibration-isolating mount I created, I never had any issues with vibrations causing issues with my sensor readings on the robot.

MPU-6050 mounted on the free floating side of the vibration isolating mount without the moon gel

The mpu-6050 mounted with the moon gel (the blue stuff). The mpu-6050 is free floated, and the opposite side is attached to the robot via Scotch external wall tape. The rubber bands ensure there is no solid attachment point to the robot to transfer vibrations easily from robot to sensor.

Early incarnation of the second major revision of a balancing robot. Note the MPU-6050 is mounted in its vibration isolating mount to the top left of the robot on the underside of the third “level”. Also note the lack of encoders on the robot, this would turn out to be a problem later.

The Arduino and PID Software:

I did a pretty thorough overview of my software in the overview section of my project, but here I wanted to add some additional details, since the heart of this project really lies in the 500-something lines of Arduino code I wrote. The main code loop has a timer to make sure the code does not run faster than 4ms, and barring that the code does not take longer than 4ms to execute, this timer ensures that the code runs as close to 4ms as possible without using interrupts. This is important because the the PID loops and most of the code on this robot relies on code being executed in constant time intervals.

For the PID algorithm I used Brett Beauregard's incredible Arduino PID library. I initially wanted to write my own algorithm from scratch but Brett already had such a capable and sophisticated algorithm it didn't make sense for me to write my own. At this point in the project, about 6 months in, I was more interested in seeing a robot balance than solve every little problem from scratch!

To make the tuning of PID algorithms faster and more flexible I made a little wired "remote" with 3 potentiometers to allow me to change the P,I, and D terms on the fly while the robot was in motion instead of having to change the values by hand and recompile the Arduino code. It also had a switch to allow me to keep the remote attached but turn "tuning mode" on and off.

Initially I toyed with the idea of placing the PID knobs onto the robot itself, but I quickly realized trying to turn these knobs while tuning a robot that was thrashing all over the place was inconvenient, and I went to the wired "remote" design. This mockup does look cool though!

The more conventional "remote". It's nothing fancy but it made tuning the PID algorithms WAY, WAY easier.

The backside of the remote with the "tuning mode" switch. When the switch was "on", the dials were active and the robot was running the experimental PID settings. When the switch was off the robot was running the last good hard-coded PID values. The sensor line of each pot and the switch gets its own wire color to easily distinguish what goes where.

Thanks to my PID remote, I came up with a rule of thumb for myself that seemed to work well for this project; if I couldn't tune the P, I and D values within 5 minutes using this remote to satisfactory results, there was probably a problem with my code somewhere. This helped me save a lot of time when trying to decide if my code was somehow holding my robot back or it was just my poor choices for P,I and D. It's easy to get stuck in the loop of trying to get perfect results out of a PID algorithm by endlessly

changing values when the PID algorithm isn't the problem. This remote and this "5 minute rule" helped me break that chain of tuning PID algorithms for hours on end.

MPU-6050 Software and Complimentary Filters:

Continuing on, regarding the code for the MPU-6050 imu, and to determine the orientation of the robot I largely used Joop Brokking's code from his amazing tutorial series on the MPU-6050. The only thing I really changed compared to his code was I removed the second complimentary filter that smoothed out the initial results from the first complimentary filter. For a balancing robot, I found this to slow down the robot's reaction adversely. I can't recommend his tutorial series enough, and I don't see any reason to deviate from the code he wrote.

However I soon discovered having just an IMU as the main sensor on the robot wasn't sufficient for a stable robot. I eventually got to the point where the robot could balance in the vertical position, but I found no matter what my PID settings were, the robot had a tendency to "run away" on smooth floors. On carpet the robot would stand up and balance with no problem, but as soon as the robot was put on a surface that didn't "push back" such as carpet, the robot almost instantly began to drift away in one direction, picking up speed until the motors couldn't keep up and the robot falls over. Initially I suspected my "I"/"integral" term on the PID algorithm was off, but no matter how much I tuned I couldn't get it to work (5 minute rule here). I looked online and people suggested adding encoders to the robot, which was a good suggestion and ended up fixing my problems. Without encoders, and just an IMU, the robot has a sense of what angle it is at with respect to the ground, but no sense of speed. Hence, if the robot is on slightly uneven ground, or starts to inevitably tip one way or the other, the wheels speed up and do their job to do balance the robot, but in the process the robot picks up momentum in one direction or another, and when the robot tries to balance itself again it adds more momentum to the robot, and the process repeats until the motors can't keep up and the robot falls over.

Adding Quadrature Encoders to the Robot:

In order to solve this problem I bought the exact same brushed DC motors I was using before from Pololu but with an added 64 CPR encoder at the motor end, which translates to 1,920 CPR at the wheel end. In theory one could have possibly also solved this problem with the accelerometer, if one kept track of every acceleration on the robot over time, the sum of all accelerations would be the velocity of the robot. In physics parlance, the integral of acceleration is velocity. I think this would be an interesting experiment for the future to see how velocity calculations from the IMU compare to the velocity measured directly from encoders.

On the left, a motor without an encoder, on the right, a motor with encoder

The standard connectors I replaced with a better quick-disconnect style plug below

Slicker quick disconnects.

Encoder Software:

I used the PJRC encoder library for the two encoders on my robot. I made use of four hardware interrupt pins on the Arduino Mega to allow each "tick" of the encoder to be picked up the moment that it happened in order to use the most accurate setting on the PJRC library. However this encoder library doesn't return the velocity of the robot, and so I derived my own solution for keeping track of how fast the robot is moving. Its nothing groundbreaking as I've stated above, but I'm proud of the fact I came up with this myself instead of just copying some tutorial :). The basic idea is velocity = (final position – initial position) / time interval of the position change. Again in physics parlance, V = delta x /

delta t (I gotta find a good wordpress plugin for equations….). Here is my entire original derivation that explains my thought process in a pretty clear fashion, even if my handwriting is messy:

My Derivation to Calculate the Velocity of the Robot:

In retrospect using the exact arc length probably wouldn't affect the balancing robot significantly as no trigonometric operations are needed as i initially assumed. However, the accuracy given to the velocity measurements with this method are so minimal it wouldn't be worth recoding to use this method.

The right side
is a
continuation
of the arc
length
method, the
left side is the
straight-line
approximation
method

Feel free to
let me
know if my
math is
wrong
somewhere
;)

The result of this math is that I can accurately measure the velocity of the robot. I employed a couple tricks with the code as well:

The Velocity Arduino Code:

```
void velocity_calculations(){

   x_final_right = right_encoder.read();
//read latest updated encoder position
    if (x_final_right != x_initial_right){
// check to make sure new encoder value is not the same as
previous one
      velocity_right = ((x_final_right -
x_initial_right)*.0114)/(.004 * time_multiple_right) ;
//delta x just change in time in seconds between encoder
readings
      x_initial_right = x_final_right;
      time_multiple_right = 1;
//if this bit of code has been run reset delta t to be
multiplied by 1

    }else{
      time_multiple_right++;
    }
```

In the above code, in the third line I check if the position of the wheel for the current encoder reading is the same as the previous encoder reading. If it is, this indicates the wheel either hasn't moved at all or is in between encoder "ticks". What I found was if I considered the velocity of the wheel to be zero at this point, the velocity of the wheel would fluctuate rapidly when in the next instant the encoder rolled over to the position, and suddenly I had a new reading and a velocity well above 0. This threw the balancing algorithms out of wack, and caused the robot to shudder violently, so in the code I just assume the velocity hasn't changed until a new encoder position is read, and keep track of how long it takes to get a new encoder reading so my delta t term is accurate. This seems to work pretty well. There is an identical piece of code here for the left encoder, I just didn't put it here.

Complimentary Filter Arduino Code:

Finally, I use a complimentary filter/ low pass high pass filter combo to smooth out the sensor readings which still changed to quickly for the PID algorithms to deal with in a smooth manner, and averaged both the velocity of the right and left encoder to give a good average velocity of the robot.

```
 filtered_velocity_left = (filtered_velocity_left * .95) +
(velocity_left * .05);            //filter the velocity output
with a low and high pass    filter to get a smooth transition
from value to value
   filtered_velocity_right = (filtered_velocity_right * .95) +
(velocity_right * .05);
   filtered_velocity_average = (filtered_velocity_left +
(filtered_velocity_right * -1) ) /2 ;
```

In retrospect, as I write this I wonder if I could have allowed a zero velocity reading from the encoders with these low pass/high pass filters? I don't actually recall if I tried this or not, as it has been over a year since this project as of me writing this, maybe in the future that could be an interesting optimization. In any case I am very happy with this encoder code and how the robot handles with encoders.

At this point, I believe the major obstacle holding back better balancing performance on my robot is better encoders and motors with less play. These pololu motors do have a noticeable amount of play if I try to rotate the wheel by hand, and while balancing in place the robot is only using a small fraction of those 1,920 counts. I could be wrong here, I'd be interested to hear what people have to say on that.

Adding a Second PID Loop:

Having encoders and being able to calculate the velocity of the robot solved the robot “run away” problem and subsequent fall when the robot was trying to balance on smooth flooring. This was because the encoders allowed the implementation of a second PID loop that commanded the first PID loop (which controlled the angle of the robot) to lean in the opposite of drift/runaway. This also came with a few benefits as a result. The fact that the robot will always fight to go to zero velocity means the robot can balance un-symmetrically loaded objects on top of it dynamically, on the fly without needing to change any PID parameters. Secondly, I can command the PID algorithm that controls velocity to go to a velocity other than zero, and the robot will automatically start to roll forward or backwards at that set speed. I wrote a little script in the code for the robot to roll forward at a commanded velocity for a set amount of time and then roll backward by using this little fact. This is how one would go about making a balancing bot remote controlled but I lost interest in the project before I got to the point of making it RC controlled.

Photo of early demonstration of non-symmetrical load carrying capability. With a…. lint roller? Maybe its just me but the robot really looks dead-set in this photo on getting that lint roller to its destination!

An Interesting Phenomenon I noticed:

Finally I did notice an interesting phenomenon with the encoder-equipped robot. I noticed that when pushed or offset, as the robot recovered from the disturbance, it didn’t just balance itself in the location where it now found itself located. The robot seemed “attracted” to back to the spot where the robot first started. I found this to be very interesting, even given a hard shove where the robot wouldn’t have a chance to naturally settle back into the spot where it started, it rolled slowly and purposefully back to its original starting location. I believe this is a result of the I, “Integral”, term on the PID algorithm dealing with velocity integrating the velocity error of the robot. Since the integral of

velocity is position, as a side effect of trying to go to a set velocity, the robot also tends to seek out its original starting position as a bonus! I thought that was really cool.

Project Conclusion:

I think that's it! If you've made it this far congratulations! I don't even know if I could've read all that, I wrote way too much! This is probably the project I am most proud of up to this point, and I wanted to share all the little technical details I thought were cool and passionate about, hence the length. I hope you enjoyed reading this as much as I enjoyed looking back on this project and reliving all my decision making and effort I put into this project!

← Previous Post Next Post →

7 thoughts on "Self-Balancing Inverted Pendulum Robot"

DAYAN
JANUARY 3, 2021 AT 12:58 PM

Grate Job , Can you more elaborate the calculation with pendulum theory

Reply

SHAY
JANUARY 4, 2021 AT 12:32 AM

I appreciate it Dayan! I'm not sure what you mean by pendulum theory, but if you are a little more specific on which portion of the calculations you would like me to elaborate further on I would be happy to try and explain it in some more detail here!

Reply

AKHILESH PRAMOD KHOT
JULY 6, 2021 AT 4:46 PM

I am working on the same project and I have read the theory regarding this.I have the state space representation of the model.
but I got stuck at programming this because representation was in continuous domain and microcontroller will need the representation in discrete domain. Can you help me with this if you followed the same approach?

Reply

SHAY SACKETT
JULY 6, 2021 AT 10:30 PM

Hello Akhilesh,

Unfortunately I don't know how much help I can offer you, because I didn't use the state-space approach for this project. I took my first controls class last semester, and while I am familiar with the state-space form, I don't have any practical experience putting the state space form into action (yet!), and my understanding of the state-space form and how to implement it in a real project is limited.

That being said, I can offer you a couple of ideas that might get you going along the right path, but again all of this is speculative. If you are purely looking to to put your equations onto a microcontroller, I would look into how to do discrete math for integrals and derivatives. The most simple way to represent a derivative in code, "dx" for example, is to set dx equal to .001, or some other tiny number, and multiply by that. To integrate, on every loop through the code, or at every "t" seconds, you recalculate the equation in question, and "sum" the latest value of the equation with the running total of all the previous iterations so that the final value of the equation is a record of the sum of all previous values of that equation.

Alternatively, I would look into using Matlab's numerous control-related toolboxes. I know they have tools that will take Matlab code (say your state-space representation) and automatically translate that into valid Arduino code, so you don't even have to worry about how to go from the continuous to discrete domains. If you already have Matlab code, this might be the simplest solution, although I've never done it myself, just throwing ideas out there based on what I know.

Finally, and you could answer this question better than I could I'm sure, but isn't the Laplace transform the tool that takes something from the continuous domain and makes it

simpler to put into the discrete domain? The Laplace transform takes all those differential and integral equations that are probably giving you trouble, and turns them all into algebraic equations, simple addition, multiplication, etc. Once the Laplace transform has everything in an algebraic form, putting those equations into a microcontroller wouldn't be a problem. It's hard for me to say without knowing all the details of your project, and again I really only have a limited theoretical knowledge of "proper" classical controls, but its another idea I'm throwing out there for you to play with. Hope this helps!

I would be interested in hearing how this project goes for you, and how you solve this problem. Feel free to shoot me a message in the contact form if you want to update me down the road, or if you want to tell me more details about what you have so far! I am interested in knowing how you derived the state-space equation for a balancing pendulum robot; I've thought about doing the same thing myself.

Reply

BILL
JANUARY 24, 2022 AT 5:15 AM

Hey, AKHILESH PRAMOD KHOT. I recommend you check out the control bootcamp videos by Steven Brunton. He shows how to design a controller in discrete time with a state-space model:
– https://www.youtube.com/playlist?list=PLMrJAkhIeNNR20Mz-VpzgfQs5zrYi085m
Look for the video about LQR control for the inverted pendulum.

Reply

HODEIFA
APRIL 9, 2022 AT 9:57 AM

Hello Shay,
Greetings from Indonesia. I am planning to make an electric unicycle based on your controller design. I notice one thing that I can't have the encoder for such large motor hub (to measure the speed and distance accurately), I need a replacement for that. What Do you suggest as the encoder alternative? Do you think the hall sensor is good enough? or Do you have other alternative idea?

Reply

G
APRIL 13, 2022 AT 4:35 PM

Thanks for sharing your experiences / lessons.

I recommend that you replace your motors with stepper motors. You won't need an encoder, you will have precise control over speed and position, and also both wheels will be in sync whereas with the motors you have now one wheel may be rotating at a slightly different speed than the other.

Reply

Leave a Comment

Your email address will not be published. Required fields are marked *

Type here..

Name*

Email*

Website

Save my name, email, and website in this browser for the next time I comment.

Post Comment »
